

Lekcja 09 - Ekosystem DE: Boto3, Avro, API

2026-08-21

```
Boto3 → S3  
fastavro → Avro  
httpx → REST API z paginacja  
PySpark vs Polars
```

■ Gdzie zyja dane w DE

W poprzednich lekcjach dane byly lokalne.

W produkcji dane sa wszedzie:

REST API	→ dane od dostawcow i partnerow
S3 / GCS	→ data lake (surowe pliki)
Kafka	→ eventy w czasie rzeczywistym
Redshift	→ data warehouse (analitka)

Python laczy te systemy.

Do kazdego jest klient.

■ Wzorzec klienta

Kazdy zewnętrzny system ma ten sam wzorzec:

```
klent      → inicjalizacja raz per sesja
operacja   → upload, download, query
blad       → specyficzny wyjatek z kodem bledu
retry      → bounded + exponential backoff + jitter
```

Nie tworzysz klienta przy kazdej operacji.

Nie ignorujesz wyjatkow.

Nie robisz retry "na slepo" dla kazdego bledu.

Juz znasz Polars. PySpark to ten sam model.

```
# Polars (lokalne)
df.filter(pl.col("status") == "completed")
df.with_columns(pl.col("amount").cast(pl.Float64))
df.group_by("status").agg(pl.sum("amount"))

# PySpark (klaster)
df.filter(df.status == "completed")
df.withColumn("amount", df.amount.cast("double"))
df.groupBy("status").agg({"amount": "sum"})
```

Roznice:

```
PySpark: lazy, klaster, withColumn (nie with_columns)
Polars:  szybki lokalnie, eager/lazy, single-node
```

Polars:

pipeline na jednej maszynie

< 100 GB

prototyp lub produkcja bez klastra

PySpark:

klaster (EMR, Databricks, GKE)

> 100 GB albo wymagany Spark w firmie

przetwarzanie równoległe na wielu nodach

Student który zna Polars uczy się PySpark w tydzień.

Model myślenia jest identyczny.

```
import boto3

# Klient – jeden per sesja
s3 = boto3.client("s3", region_name="eu-west-1")

# Upload pliku
s3.upload_file("output/orders.parquet", "my-bucket", "
orders/orders.parquet")

# Download pliku
s3.download_file("my-bucket", "orders/orders.parquet", "
/tmp/orders.parquet")

# Upload bajtow (bez pliku posredniego)
s3.put_object(Bucket="my-bucket", Key="orders/data.avro", Body
=avro_bytes)
```

```
from botocore.exceptions import ClientError

try:
    s3.download_file(bucket, key, local_path)
except ClientError as exc:
    code = exc.response["Error"]["Code"]
    if code == "NoSuchKey":
        raise FileNotFoundError(f"Not found: s3://{bucket}/{key}")
    from exc
    raise # rethrow nieznane błędy
```

Wzorzec:

```
ClientError ma kod błędu w response dict.
"NoSuchKey" → plik nie istnieje.
"AccessDenied" → problem z uprawnieniami.
Inne → rethrow.
```

■ Avro – format streamingowy

Parquet i Avro to obydwa binary formats ze schema.

Ale używane w innych miejscach:

Parquet:

columnar – świetny do query (SELECT kilka kolumn)
batch analytics, data lake, Redshift, BigQuery

Avro:

row-based – świetny do write (jeden event naraz)
Kafka messages, CDC, API events

Jesli dane ida przez Kafka – prawdopodobnie Avro.

Jesli dane laduja w data lake do query – Parquet.

```
import fastavro
from io import BytesIO

SCHEMA = {
    "type": "record",
    "name": "Order",
    "fields": [
        {"name": "order_id", "type": "string"},
        {"name": "status", "type": "string"},
        {"name": "total_amount", "type": "double"},
    ],
}

# Zapis
records = [{"order_id": "1001", "status": "completed", "
total_amount": 120.5}]
buf = BytesIO()
fastavro.writer(buf, fastavro.parse_schema(SCHEMA), records)

# Odczyt
buf.seek(0)
loaded = list(fastavro.reader(buf))
```

Wiele API nie zwraca wszystkich rekordów naraz.

Wzorzec cursor-based:

```
def fetch_all(base_url: str, client: httpx.Client) ->
    list[dict[str, Any]]:
    all_records: list[dict[str, Any]] = []
    next_cursor: str | None = None

    while True:
        params = {"cursor": next_cursor} if next_cursor else {}
        response = client.get(f"{base_url}/orders", params=params)
        response.raise_for_status()          # rzuca jesli 4xx/5xx
        data = response.json()
        all_records.extend(data["results"])
        next_cursor = data.get("next")       # None → ostatnia

    strona

    if not next_cursor:
        break

    return all_records
```

■ Kiedy retry ma sens

Retry tylko dla błędów przejściowych:

```
429 (rate limit)
408 (timeout)
5xx (chwilowa awaria serwera)
```

Bez retry dla błędów stałych:

```
400 (zły request)
401/403 (auth)
404 (zły endpoint)
```

■ Exponential backoff

Nie spamujesz API co 100 ms.

Wzorzec opoznien:

```
attempt 1 -> 0.5s  
attempt 2 -> 1.0s  
attempt 3 -> 2.0s  
attempt 4 -> 4.0s
```

To zmniejsza presje na system po drugiej stronie.

■ Jitter (dlaczego)

Bez jittera wiele workerow retryuje w tej samej sekundzie.

```
thundering herd
```

Z jitterem kazdy worker czeka odrobine inaczej.

```
delay = base_delay * 2**attempt + losowy_komponent
```

```
def retryable_get(client: httpx.Client, url: str, params:
dict[str, Any]) -> httpx.Response:
    max_attempts = 4
    base_delay = 0.5

    for attempt in range(max_attempts):
        response = client.get(url, params=params)
        if response.status_code in {429, 500, 502, 503, 504} and
attempt < max_attempts - 1:
            jitter = random.uniform(0, 0.2)
            delay = base_delay * (2 ** attempt) + jitter
            time.sleep(delay)
            continue
        response.raise_for_status()
        return response

    raise RuntimeError("unreachable")
```

■ Idempotency i retry

```
GET -> zwykle bezpieczny retry  
PUT -> zwykle bezpieczny retry  
POST -> retry tylko z idempotency key
```

W DE to kluczowe, bo duplikaty psuja metryki.

■ Mocking w testach

Zewnętrzne systemy (S3, API) nie powinny być wywoływane w testach.

```
from unittest.mock import MagicMock

def test_upload_calls_put_object() -> None:
    mock_s3 = MagicMock()
    upload_to_s3(b"data", bucket="my-bucket", key="k", client
=mock_s3)
    mock_s3.put_object.assert_called_once_with(
        Bucket="my-bucket", Key="k", Body=b"data"
    )
```

Klient jako parametr = testowalność bez prawdziwego AWS.

Klient w środku funkcji = nie da się przetestować bez moto/localstack.

Pytanie:

How do you handle external integrations in a Python DE pipeline?

Odpowiedz:

I treat each external system as a boundary.
The client is created once per session, not per operation.
IO functions accept the client as a parameter – this makes them testable.
In tests I replace real clients with MagicMock and verify the call arguments.
For S3 I use boto3 with ClientError handling.
For REST APIs I use httpx with cursor-based pagination and bounded retries.
Retry is only for transient errors with exponential backoff and jitter.

Oddajesz:

```
src/ingestion/  
  api.py      ← fetch z paginacja + retry/backoff  
  serialize.py ← Avro roundtrip  
  storage.py  ← upload z klientem jako parametr  
tests/  
  test_api.py  
  test_serialize.py  
  test_storage.py
```

Najwazniejsze:

```
testy nie robia prawdziwych HTTP ani S3 calls  
retry dziala tylko dla transient statusow  
mypy src/ – czysto
```